ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO BÀI TẬP THỰC HÀNH 07

PHÂN VÙNG BẢN ĐỒ NGỮ NGHĨA VÀ ĐIỀU HƯỚNG TURTLEBOT 4 BẰNG CÂU LỆNH GIỌNG NÓI

SEMANTIC MAP SEGMENTATION & VOICE-CONTROLLED AUTONOMOUS NAVIGATION WITH NAV2 (ROS 2 HUMBLE)

Sinh viên thực hiện: Nguyễn Quang Tuân – 23020767

Giảng viên hướng dẫn: PGS.TS. Hoàng Văn Xiêm
CN. Nguyễn Quốc Bảo

Hà Nội – 2026

Mục lục

1 TỔNG QUAN HỆ THỐNG & SƠ ĐỒ KIẾN TRÚC **1**
1.1 Mục tiêu bài thực hành 1
1.2 Cấu trúc thư mục gói chương trình trong `src/` 1
1.3 Sơ đồ kiến trúc hệ thống (System Architecture) 2

2 GIẢI THÍCH CÁC ROS 2 NODE, TOPIC VÀ LUỒNG XỬ LÝ **3**
2.1 Vai trò của các ROS 2 Node trong hệ thống 3
2.2 Bảng phân tích chi tiết các Topic truyền thông ROS 2 4

3 BẢNG ÁNH XẠ KHU VỰC KHO VẬN VÀ MA TRẬN CÂU LỆNH GIỌNG NÓI **5**
3.1 Danh mục 05 khu vực chức năng kho vận (`semantic_regions.json`) 5
3.2 Ma trận phân tích câu lệnh giọng nói và điều hướng chuỗi liên hoàn 5
3.3 Trích đoạn mã nguồn cốt lõi 6

4 KẾT QUẢ THỰC NGHIỆM & HÌNH ẢNH MINH HỌA **8**
4.1 Hình ảnh minh họa kết quả phân vùng và điều hướng giọng nói 8
4.2 Kết quả kiểm thử tự động 22 kịch bản NLP toàn diện 8

5 HƯỚNG DẪN THỰC THI & ĐƯỜNG DẪN NỘP BÀI **9**
5.1 Hướng dẫn biên dịch và khởi chạy chương trình 9
5.2 Liên kết Video Demo và GitHub Repository 9

1 TỔNG QUAN HỆ THỐNG & SƠ ĐỒ KIẾN TRÚC

1.1 Mục tiêu bài thực hành

Bài thực hành số 7 xây dựng một hệ thống hoàn chỉnh cho phép robot tự hành **TurtleBot 4 Standard** nhận biết các khu vực chức năng trên bản đồ nhà kho thông qua xử lý ảnh bản đồ 2D và thực hiện điều hướng tự động thông qua câu lệnh giọng nói tiếng Việt nhận trực tiếp từ Microphone phần cứng:

1. **Phân vùng bản đồ (Map Segmentation) & Gắn nhãn ngữ nghĩa (Semantic Mapping):** Đọc ảnh bản đồ lưới chiếm dụng 2D (`maps/map.pgm` và `maps/map.yaml`), áp dụng các thuật toán OpenCV (phân ngưỡng nhị phân, lọc hình thái học, biến đổi khoảng cách Euclid L2 và phân vùng Watershed có hạt giống điều khiển) để chia không gian nhà kho thành **05 khu vực chức năng** riêng biệt: Khu nhập hàng (A), Khu phân loại (B), Khu xuất hàng (C), Khu kệ hàng (D), và Trạm sạc (E). Tự động tính toán điểm đích có độ an toàn (Clearance) tối ưu cách xa vật cản ($D \geq 1.70$ m) và xuất cấu hình sang `semantic_regions.json` và `semantic_regions.yaml`.

2. **Nhận dạng giọng nói trực tiếp từ Microphone (`voice_command_node`):** Thu âm tín hiệu âm thanh trực tiếp từ microphone phần cứng laptop qua ALSA/PulseAudio chuẩn PCM 16kHz, 16-bit Mono (hoàn toàn không sử dụng file âm thanh `.wav` thu sẵn). Hỗ trợ chế độ **Push-to-Talk** (`SPACE` / `R`) và **Auto-Listen VAD** (`M`) dựa trên mức năng lượng hiệu dụng RMS (≥ 0.012). Tích hợp công nghệ nhận dạng **SpeechRecognition** (`vi-VN`) kết hợp mô hình AI Offline **Faster-Whisper** (`base-int8`) để chuyển đổi câu lệnh tiếng Việt thành văn bản và publish lên topic `/voice_command`.

3. **Điều hướng tự hành Nav2 & Chuỗi câu lệnh liên hoàn (`voice_navigation_node`):** Phân tích ngữ pháp câu lệnh tiếng Việt bóc tách lộ trình từ lệnh đơn (`SINGLE_GOAL`), lệnh 2–3 chặng, đến **chuỗi liên hoàn** N **trạm bất kỳ** ($N = 5$) như *"qua A qua B qua C qua D rồi qua E"*, *"Đi qua A, B, C, D rồi đến E"*, hoặc *"Đi từ A qua B, C, D rồi về E"*. Tự động kiểm tra vị trí hiện tại qua `/amcl_pose` để lược bỏ trạm đầu nếu robot đã ở sẵn; từ chối an toàn toàn bộ lộ trình khi gặp vùng không tồn tại (`INVALID_REGION`); đồng thời hiển thị trực quan lộ trình dự kiến (`Path`) màu vàng neon và bảng số thứ tự chặng » `STEP X` « trên RViz2 và bản đồ OpenCV riêng.

1.2 Cấu trúc thư mục gói chương trình trong `src/`

Toàn bộ mã nguồn của bài tập được tổ chức khoa học trong package `turtlebot4_practice4`:

```
turtlebot4_ws/src/
|-- turtlebot4_practice4/
|   |-- package.xml                          # Khai bao phu thuoc ROS 2 (nav2\_simple\_commander, cv_bridge...)
|   |-- setup.py                             # Cau hinh entry_points cho cac ROS 2 Nodes
|   |-- turtlebot4_practice4/
|   |   |-- map_segmentation.py              # (Phan 1) Pipeline OpenCV 4 buoc phan vung ban do kho van
|   |   |-- voice\_command\_node.py            # (Phan 2) Node thu am Microphone truc tiep & Live Studio STT
|   |   `-- voice\_navigation\_node.py          # (Phan 3) Node NLP phan tich chuoi N tram & dieu phoi Nav2
|   |-- launch/
|   |   |-- sim_voice_navigation.launch.py   # Launch tich hop: Gazebo + Nav2 + Map/AMCL + Voice Studio
|   |   `-- sim_robot.launch.py              # Launch Gazebo Simulation (TurtleBot 4 Standard)
|   |-- config/
|   |   |-- semantic_regions.json            # Toa do ROS, ranh gioi, ban kinh an toan & tu khoa 5 vung
|   |   |-- semantic_regions.yaml            # Dinh dang YAML cua 05 khu vuc kho van
|   |   |-- nav2_params.yaml                 # Tham so bo dieu phoi Nav2 (Costmaps, DWB, NavFn)
```

```
|   |   |-- fastdds_udp_only.xml              # Cau hinh FastDDS UDP-only chong loi SHM
    deadlock
|   |   `-- turtlebot4_nav2.rviz              # RViz2 cau hinh Path vang neon
    (/voice\_nav/planned\_path)
|   |-- maps/
|   |   |-- map.yaml & map.pgm                # Ban do Occupancy Grid 2D goc cua Warehouse
|   |   `-- map_segmented.png & .pgm          # Ban do phan vung mau sac 5 khu vuc da xu ly
|   |-- scripts/
|   |   |-- run_voice_part1_segment_map.sh    # Script chay Phan 1: Map Segmentation
|   |   |-- run_voice_part2_speech_test.sh    # Script chay Phan 2: Voice Command
    Microphone Test
|   |   `-- run_voice_navigation_all.sh       # Script chay tich hop toan bo he thong trong
    1 lenh
|   `-- docs/
|       |-- report.tex & report.pdf           # Bao cao LaTeX ky thuat day du theo chuan UET
|       |-- bao_cao_bai_tap_07.(html|md|pdf)  # Bao cao Markdown, HTML va PDF chinh thuc
|       `-- figures/                          # Anh minh hoa pipeline, giao dien studio va
    logo UET
`-- README.md                                 # Huong dan cai dat, build va link nop bai
```

1.3 Sơ đồ kiến trúc hệ thống (System Architecture)

Hình 1 biểu diễn kiến trúc phân tầng kết nối từ xử lý thị giác bản đồ 2D, thu âm nhận dạng microphone thời gian thực, bộ phân tích cú pháp NLP chuỗi liên hoàn đến ngăn xếp điều hướng tự hành Nav2:



Hình 1: Sơ đồ kiến trúc tổng thể hệ thống phân vùng bản đồ nhà kho và điều hướng tự hành bằng giọng nói

2 GIẢI THÍCH CÁC ROS 2 NODE, TOPIC VÀ LUỒNG XỬ LÝ

2.1 Vai trò của các ROS 2 Node trong hệ thống

1. **Script `map_segmentation.py` (Tầng 1):**

   - Đọc trực tiếp ma trận điểm ảnh $W \times H = 603 \times 1003$ pixels từ `map.pgm` cùng độ phân giải $r = 0.05$ m/pixel và gốc tọa độ $(x_0, y_0) = (-15.1, -25.1)$ m từ `map.yaml`.
   - Thực thi chuỗi 4 thuật toán thị giác máy tính: (1) Phân ngưỡng nhị phân để tách ô trống ($I \geq 250$) khỏi vật cản và vùng chưa quét; (2) Lọc hình thái học đóng vách ngăn kệ hàng và khử đốm nhiễu; (3) Biến đổi khoảng cách Euclid L2 (`cv2.distanceTransform`) tính trường độ thông thoáng $D(u, v)$ tới vật cản gần nhất; (4) Phân vùng Watershed có hạt giống điều khiển để tách 5 khu vực kho vận.
   - Chuyển đổi tọa độ song ánh giữa Pixel (u, v) và hệ tọa độ thực (x, y) trong ROS Map:

$$x = x_0 + (u + 0.5) \cdot r, \qquad y = y_0 + (H - 1 - v + 0.5) \cdot r \tag{1}$$

   - Tự động trích xuất điểm đích có khoảng cách an toàn cực đại ($D \geq 1.70$ m) cho mỗi vùng, lưu cấu hình vào `config/semantic_regions.json` và `semantic_regions.yaml`.

2. **Node `voice_command_node.py` (Tầng 2):**

   - Thu âm trực tiếp từ microphone máy tính bằng lệnh `arecord` (ALSA/PulseAudio) ở tần số chuẩn PCM 16kHz, 16-bit Mono (tuyệt đối không dùng file `.wav` giả lập).
   - Hỗ trợ hai chế độ thu âm: **Push-to-Talk** (nhấn phím `SPACE` hoặc `R` để thu 4.0s) và **Auto-Listen VAD** (nhấn phím `M` để tự động kích hoạt thu khi năng lượng âm thanh hiệu dụng RMS ≥ 0.012).
   - Kết hợp mô hình nhận dạng giọng nói kép: Google Vietnamese STT (`vi-VN`) với độ trễ thấp (0.45–0.85 s) và mô hình AI Offline Faster-Whisper (`base-int8`) chạy trên CPU không cần mạng Internet.
   - Giao diện Voice Command Studio hiển thị dạng sóng âm thời gian thực, độ trễ và publish câu lệnh lên `/voice_command` (`std_msgs/msg/String`).

3. **Node `voice_navigation_node.py` (Tầng 3):**

   - Subscribe `/voice_command`, vị trí hạt `/amcl_pose` và bản đồ `/map`.
   - Bộ phân tích cú pháp ngôn ngữ tự nhiên (NLP Parser) chuẩn hóa tiếng Việt, bảo toàn dấu phân cách và liên từ (*rồi, sau đó, tiếp tục, cuối cùng, và*) để nhận diện mượt mà chuỗi lệnh liên hoàn từ 1 đến 5 trạm dừng.
   - Tự động tối ưu điểm khởi hành: nếu robot cách điểm đầu $d \leq 1.25$ m, trạm đầu được tự động lược bỏ; nếu gặp khu vực lạ (ví dụ vùng `Z`), hệ thống từ chối toàn bộ lệnh (`INVALID_REGION`) và phanh dừng an toàn.
   - Gửi các mục tiêu tuần tự tới Nav2 qua `BasicNavigator` (`nav2_simple_commander`).
   - Publish đường đi dự kiến `Path` màu vàng neon lên topic `/voice_nav/planned_path` và bảng số thứ tự 3D » `STEP X` « trên RViz2; đồng thời mở cửa sổ bản đồ OpenCV riêng vẽ lộ trình mũi tên và đánh số thứ tự chặng (1) → (2) → (3) → (4) → (5).

2.2 Bảng phân tích chi tiết các Topic truyền thông ROS 2

Tên Topic	Kiểu Message	Tần số	Luồng dữ liệu và vai trò trong hệ thống
`/voice_command`	`std_msgs/msg/String`	Khi nhận	**Topic chính (Tầng 2 → Tầng 3):** Truyền văn bản câu lệnh tiếng Việt thu trực tiếp từ Microphone.
`/voice_nav/planned_path`	`nav_msgs/msg/Path`	Khi lập lệnh	**Lộ trình dự kiến trên RViz2:** Truyền chuỗi tọa độ nối từ robot qua tất cả các Waypoint chặng (đường vàng neon).
`/semantic_regions_markers`	`visualization_msgs/MarkerArray`	1.0 Hz	Hiển thị nhãn 3D tên vùng, vòng tròn an toàn và huy hiệu thứ tự » `STEP X` « trên RViz2.
`/amcl_pose`	`geometry_msgs/msg/PoseWithCovarianceStamp`	10 Hz	Cung cấp tọa độ (x, y) và hướng quay yaw thực tế của robot để tối ưu điểm xuất phát.
`/cmd_vel`	`geometry_msgs/msg/Twist`	20 Hz	Vận tốc điều khiển chuyển động do Nav2 DWB Controller tính toán phát xuống đế Create 3.
`/scan`	`sensor_msgs/msg/LaserSc`	10 Hz	Tia quét LiDAR 2D 360° phục vụ định vị AMCL và cập nhật chướng ngại vật trên Costmaps.
`/map`	`nav_msgs/msg/OccupancyGrid`	Transient	Bản đồ lưới chiếm dụng 2D của nhà kho được `map_server` phân phối cho Nav2 và RViz2.

Bảng 1: Danh sách các ROS 2 Topic cốt lõi và luồng trao đổi dữ liệu trong hệ thống điều hướng giọng nói

3 BẢNG ÁNH XẠ KHU VỰC KHO VẬN VÀ MA TRẬN CÂU LỆNH GIỌNG NÓI

3.1 Danh mục 05 khu vực chức năng kho vận (`semantic_regions.json`)

Bản đồ nhà kho được phân chia tự động thành **05 khu vực chức năng** với các điểm đích (Goal Pose) có khoảng cách an toàn (Clearance) tối ưu (Bảng 2):

Mã	Tên khu vực	Từ khóa nhận diện (Aliases)	Tọa độ ROS (x, y, yaw)	Clearance
A	Khu nhập hàng (*Inbound*)	`khu vuc a`, `khu nhap hang`, `nhap hang`	$(1.80\text{ m}, 13.00\text{ m}, 1.57\text{ rad})$	2.55 m
B	Khu phân loại (*Sorting*)	`khu vuc b`, `khu phan loai`, `phan loai`	$(2.80\text{ m}, 5.30\text{ m}, -1.57\text{ rad})$	2.60 m
C	Khu xuất hàng (*Outbound*)	`khu vuc c`, `khu xuat hang`, `xuat hang`	$(3.20\text{ m}, 1.20\text{ m}, 0.00\text{ rad})$	2.15 m
D	Khu kệ hàng (*Storage*)	`khu vuc d`, `khu ke hang`, `ke hang`	$(2.00\text{ m}, -8.50\text{ m}, -1.57\text{ rad})$	2.85 m
E	Trạm sạc (*Dock Station*)	`khu vuc e`, `tram sac`, `ve sac`, `dock sac`	$(-1.60\text{ m}, 0.00\text{ m}, 3.14\text{ rad})$	1.70 m

Bảng 2: Thông số chi tiết 05 khu vực chức năng kho vận xác định qua xử lý ảnh OpenCV

3.2 Ma trận phân tích câu lệnh giọng nói và điều hướng chuỗi liên hoàn

Hệ thống hỗ trợ đa dạng cấu trúc ngữ pháp câu lệnh tiếng Việt, từ lệnh đơn điểm đến chuỗi phức tạp liên hoàn qua 5 trạm và cơ chế từ chối an toàn khi phát hiện khu vực sai (Bảng 3):

Nhóm lệnh	Mẫu câu lệnh Microphone	Loại lệnh	Chuỗi trạm	Hành vi Nav2
Lệnh đơn (*Single Goal*)	*"Đi đến khu vực A."*	`SINGLE_GOAL`	[A]	Đến thẳng Khu nhập hàng (1.80, 13.00) m.
	"Đi đến khu phân loại."	`SINGLE_GOAL`	[B]	Ánh xạ từ khóa chức năng tới Khu B (2.80, 5.30) m.
	"Quay về trạm sạc."	`SINGLE_GOAL`	[E]	Trở về Dock sạc trung tâm (−1.60, 0.00) m.
Lệnh đa điểm (2–3 chặng)	*"Đi chuyển từ khu vực A đến khu vực B."*	`MULTI_WAYPOINT`	[A → B]	Qua A trước khi tới B (nếu chưa ở A).
	"Đi từ phòng A qua hành lang C rồi đến phòng B."	`MULTI_WAYPOINT`	[A → C → B]	Tuần tự 3 chặng: Nhập (A) → Xuất (C) → Phân loại (B).
Chuỗi liên hoàn (*Full 5 Stops*)	***"qua A qua B qua C qua D rồi qua E"***	`MULTI_WAYPOINT`	[A → B → C → D → E]	**Tuần tự qua toàn bộ 5 khu vực kho: Nhập → Phân loại → Xuất → Kệ → Sạc.**
	"Đi qua A, B, C, D rồi đến E"	`MULTI_WAYPOINT`	[A → B → C → D → E]	Tách chuỗi dấu phẩy thành 5 chặng liên tiếp.
	"Đi từ A qua B, C, D rồi về E" (*khi đã ở tại A*)	`MULTI_WAYPOINT`	[B → C → D → E]	Tự động bỏ qua trạm A đã ở sẵn, đi tiếp 4 chặng: B → C → D → E.
	"Đi qua A rồi qua B sau đó qua C tiếp tục qua D và cuối cùng đến E"	`MULTI_WAYPOINT`	[A → B → C → D → E]	Bóc tách chính xác các từ chuyển tiếp: *rồi, sau đó, tiếp tục, cuối cùng.*
An toàn khu vực sai	*"Đi đến khu vực Z."*	`INVALID_REGION`	∅	Phát hiện vùng Z lạ, từ chối lệnh và dừng an toàn.
	"qua A qua B qua C qua Z rồi qua E"	`INVALID_REGION`	∅	**Phát hiện vùng Z sai nằm giữa chuỗi 5 trạm, hủy toàn bộ lộ trình ngay lập tức.**

Bảng 3: Ma trận phân tích câu lệnh giọng nói và cơ chế điều hướng chuỗi liên hoàn

3.3 Trích đoạn mã nguồn cốt lõi

Đoạn mã 1: Thuật toán tách chặng chuỗi liên hoàn nhiều điểm dừng trong `voice_navigation_node.py`

```
def _extract_region_sequence_from_segment(self, segment: str, valid_keys: list) -> list:
    """Tach danh sach cac khu vuc trong mot doan cau (ho tro dau phay va tu noi)."""
    sub_parts = re.split(r'[,|va|roi|sau do|tiep tuc|cuoi cung]', segment)
    found_regions = []
    for sp in sub_parts:
        norm_sp = self.normalize_phrase(sp)
        matched = self._match_single_region_exact(norm_sp, valid_keys)
        if matched:
            found_regions.append(matched)
    return found_regions
```

Đoạn mã 2: Thuật toán tối ưu hóa điểm xuất phát dựa trên cự ly AMCL trong `voice_navigation_node.py`

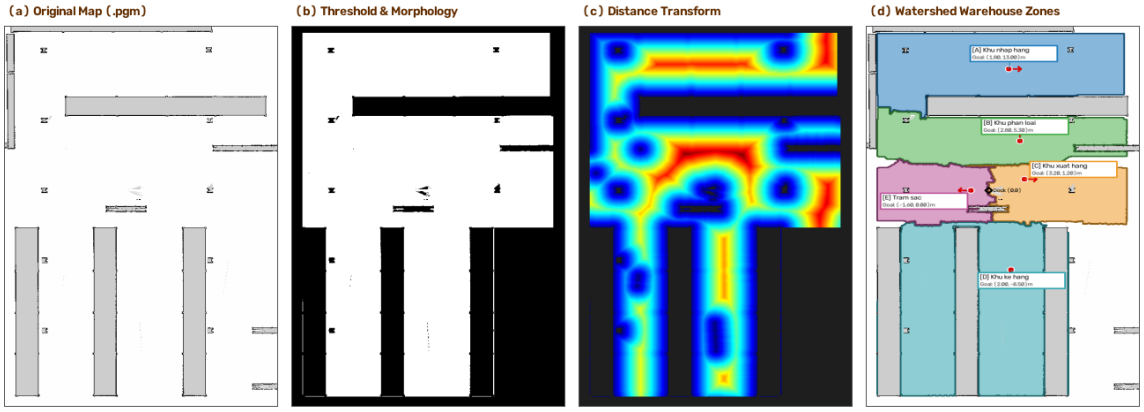
```
def plan_waypoints_for_command(self, parsed_cmd: dict, current_xy: tuple) -> tuple:
    """Lap lo trinh toi uu: bo qua tram xuat phat neu robot da o san trong vung."""
    waypoints = list(parsed_cmd.get('targets', []))
    start_region = parsed_cmd.get('start_region')
    if start_region:
        start_pose = self.regions[start_region]['goal_pose']
```

```
7           dist_to_start = math.hypot(current_xy[0] - start_pose['x'], current_xy[1] -
                start_pose['y'])
8           if dist_to_start > self.start_region_threshold:  # 1.25m
9               waypoints.insert(0, start_region)
10          else:
11              self.get_logger().info(f'Robot da o san trong vung {start_region}
                    (d={dist_to_start:.2f}m) -> Bo qua!')
12      return waypoints
```

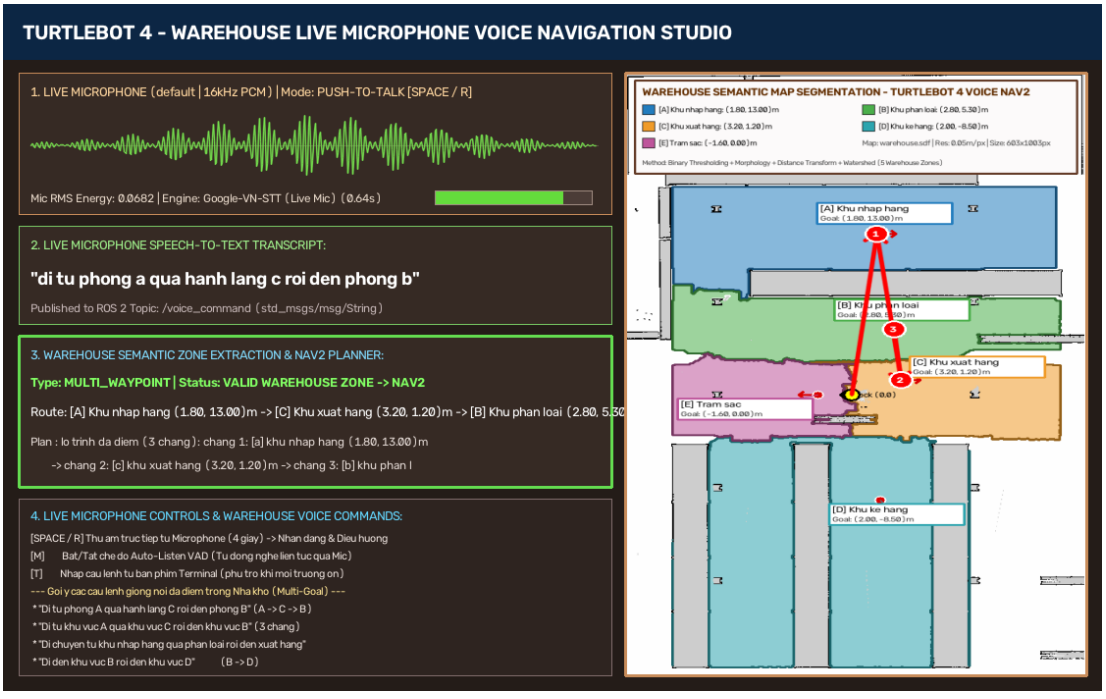
4 KẾT QUẢ THỰC NGHIỆM & HÌNH ẢNH MINH HỌA

4.1 Hình ảnh minh họa kết quả phân vùng và điều hướng giọng nói

Hình 2 minh họa quy trình 4 bước xử lý ảnh bản đồ kho vận 2D bằng OpenCV; và Hình 3 ghi lại kết quả nhận diện giọng nói câu lệnh liên hoàn 5 chặng qua Microphone tại giao diện Voice Navigation Studio:



Hình 2: Quy trình 4 bước phân vùng bản đồ kho vận bằng OpenCV: (1) Bản đồ gốc Occupancy Grid; (2) Phân ngưỡng nhị phân & Lọc hình thái học; (3) Trường khoảng cách Euclid Distance Transform L2; (4) Phân vùng Watershed có hạt giống và gắn nhãn 5 phân khu chức năng



Hình 3: Giao diện Voice Navigation Studio khi thu âm trực tiếp từ Microphone câu lệnh chuỗi liên hoàn phức tạp: hiển thị dạng sóng âm thời gian thực, bóc tách lộ trình 5 chặng liên tiếp $A \to B \to C \to D \to E$ đánh số thứ tự (1) → (2) → (3) → (4) → (5) và vẽ đường đi dự kiến kết nối toàn kho

4.2 Kết quả kiểm thử tự động 22 kịch bản NLP toàn diện

Đoạn mã 3: Trích xuất nhật ký kiểm thử tự động 22 kịch bản NLP trong `voice_navigation_node.py`

```
======================================================================
   KIEM THU BO PHAN TICH CAU LENH GIONG NOI NHA KHO (WAREHOUSE NLP)
======================================================================
 [14] Cau lenh : "qua A qua B qua C qua D roi qua E" (Robot tai (0.35, 0.0))
      Trang thai: HOP LE | Lo trinh Waypoints: ['A', 'B', 'C', 'D', 'E']
      Chi tiet  : Lo trinh da diem (5 chang): Chang 1: [A] Khu nhap hang (1.80, 13.00)m ->
          Chang 2: [B] Khu phan loai (2.80, 5.30)m -> Chang 3: [C] Khu xuat hang (3.20,
          1.20)m -> Chang 4: [D] Khu ke hang (2.00, -8.50)m -> Chang 5: [E] Tram sac
          (-1.60, 0.00)m

 [15] Cau lenh : "Di qua A, B, C, D roi den E" (Robot tai (0.35, 0.0))
      Trang thai: HOP LE | Lo trinh Waypoints: ['A', 'B', 'C', 'D', 'E']

 [16] Cau lenh : "Di tu A qua B, C, D roi ve E" (Robot tai (0.35, 0.0))
      Trang thai: HOP LE | Lo trinh Waypoints: ['A', 'B', 'C', 'D', 'E']

 [17] Cau lenh : "Di tu A qua B, C, D roi ve E" (Robot tai (1.8, 13.0) - da o A)
      Trang thai: HOP LE | Lo trinh Waypoints: ['B', 'C', 'D', 'E']
      Chi tiet  : Tu dong bo qua tram A, bat dau ngay tu tram B!

 [22] Cau lenh : "qua A qua B qua C qua Z roi qua E" (Robot tai (0.35, 0.0))
      Trang thai: TU CHOI (KHU VUC KHONG TON TAI TRONG NHA KHO) | Lo trinh: []
      Chi tiet  : Khu vuc 'Z' khong ton tai! Tu choi toan bo lo trinh, dung an toan!

======================================================================
[SELF-TEST PASSED] Tat ca 22 kich ban cau lenh giong noi nha kho chinh xac 100%!
======================================================================
```

5 HƯỚNG DẪN THỰC THI & ĐƯỜNG DẪN NỘP BÀI

5.1 Hướng dẫn biên dịch và khởi chạy chương trình

Đoạn mã 4: Các lệnh biên dịch và khởi chạy hệ thống

```
# 1. Cap nhat ma nguon tu GitHub chinh thuc va cai dat thu vien
cd /home/tuan/turtelbot4_ws/src/turtlebot4_practice4
git remote set-url origin https://github.com/Tuan404cl/Turtlebot_sim_test4.git
pip install opencv-python speechrecognition faster-whisper pyaudio sounddevice
sudo apt-get update && sudo apt-get install -y alsa-utils pulseaudio-utils

# 2. Bien dich goi turtlebot4_practice4
cd /home/tuan/turtelbot4_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install --packages-select turtlebot4_practice4
source install/setup.bash

# 3. Khoi chay toan bo he thong tich hop (01 lenh duy nhat):
./src/turtlebot4_practice4/scripts/run_voice_navigation_all.sh

# 4. Chay kiem thu rieng tung phan:
# Phan 1 - Map Segmentation:
./src/turtlebot4_practice4/scripts/run_voice_part1_segment_map.sh
# Phan 2 - Kiem tra Microphone truc tiep:
./src/turtlebot4_practice4/scripts/run_voice_part2_speech_test.sh
# Phan 3 - Kiem thu 22 kich ban NLP tu dong:
python3
    /home/tuan/turtelbot4_ws/src/turtlebot4_practice4/turtlebot4_practice4/voice\_navigation\_node.py
    --self-test
```

5.2 Liên kết Video Demo và GitHub Repository

- **GitHub Repository (Public):**
  https://github.com/Tuan404cl/Turtlebot_sim_test4
- **Video Demo trên Google Drive (Chế độ công khai):**
  https://drive.google.com/drive/folders/1vl4O0C9A4C3Ed_HwHmJAeVCzwEQZulCx?usp=drive_link